
Exercise: Multi-Role JWT Auth for a Fake API

Ask students to build a single Python script that simulates a tiny backend with users, passwords, and JWT-based access control.

Step 1 – User “Database” + Signup

1. Create an in-memory “users table” as a Python dict:

```
users = {} # key = username, value = { "password_hash": ...,
    "role": ... }
```

1. Implement a function `signup(username: str, password: str, role: str)` that:
 - Hashes the password using `bcrypt.hashpw`.
 - Stores `password_hash` and `role` in `users`.
 - Prints the stored hash and role.
2. Test by creating at least two users:
 - `alice`, password `"alice123"`, role `"admin"`
 - `bob`, password `"bob123"`, role `"user"`

(We are reusing the hashing logic from `demo.py`, but now per user.)

Step 2 – Login and JWT Issuance

1. Implement `login(username: str, password: str) -> str | None` that:
 - Looks up the user from `users`.
 - Verifies the password using `bcrypt.checkpw`.
 - If correct, builds a JWT **payload** with:

```
{
  "sub": username,
  "role": user_role,
```

```
"exp": int(time.time()) + 60 # 60 seconds expiry
}
```

- Signs it using the same `SECRET\_KEY` and `ALGO` pattern as in `demo.py`.[^1]
- Returns the token (string), or `None` if login fails.

1. Print the generated token for alice and bob.

(This reinforces: hashes vs passwords, and that JWT payload is not encrypted.)

Step 3 – Token Validation + Authorization Decorator

1. Implement a function `decode_token(token: str)` that:
 - Uses `jwt.decode` with `SECRET_KEY` and `ALGO`.
 - Handles:
 - `ExpiredSignatureError` → print “Token expired”.
 - `InvalidTokenError` → print “Invalid token”.
 - Returns the decoded payload dict if valid, or `None` otherwise.[^1]
2. Implement a helper:

```
def require_role(token: str, allowed_roles: list[str]) -> dict |  
    None:  
    # returns decoded payload if user has required role, else None
```

- Internally calls `decode_token`.
- Checks if `payload["role"]` is in `allowed_roles`.
- Prints “Access granted” or “Access denied” accordingly.
- Returns payload on success, `None` on failure.

(This makes us explicitly separate **authentication** – token valid – from **authorization** – role allowed.)

Step 4 – Fake Protected Endpoints

Simulate two “API endpoints” as plain functions:

1. `delete_user(token, username_to_delete)`:
 - Only admin role allowed.
 - Use `require_role(token, ["admin"])`.

- If allowed, print `f"User {username_to_delete} deleted"` (no real delete needed).
 - Else, print `"Forbidden"`.
2. `view_profile(token)`:
- Both `"admin"` and `"user"` can access.
 - Use `require_role(token, ["admin", "user"])`.
 - If allowed, print `f"Showing profile for {payload['sub']}"`.

Test cases you can run now:

- Login as alice (admin) and call both endpoints.
 - Login as bob (user) and try to call `delete_user`.
 - Wait >60 seconds and call any endpoint again with the old token to see expiry.
-